Implementation Document

for

Centralized Mess Management System (CMMS)

Version 1.0

Prepared by

Group #: 19 **Group Name: Full Throttle Tenacious Dynasty**

Aaditya Bijarniya	230004	adityab23@iitk.ac.in
Tanush Khagwal	231083	tanushk23@iitk.ac.in
Kavita Kumari	230552	kavitak23@iitk.ac.in
Soumyaranjan Sahu	241035	soumyasahu24@iitk.ac.in
Deepak Kumar	240329	deepakk24@iitk.ac.in
Ashwina Yadav	230235	ashwinay23@iitk.ac.in
Itesh Mishra	230489	iteshm23@iitk.ac.in
Shubham Kumar Patel	241010	shubhamkp24@iitk.ac.in
Dev Prakash	240338	devp24@iitk.ac.in
Shital Niras	230967	shitalan23@iitk.ac.in

Course: CS253

Mentor TA: Vinayak Bhosle

Date: 27 March 2026

CONTENTS

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Draft Type and Number	Aaditya Bijarniya Shubham Kumar Dev Prakash Shital Anurath Niras Kavita Kumari Tanush Khagwal Ashwina Yadav Deepak Kumar Soumyaranjan Sahu	First Draft of Implementation document of the project CMMS	27/03/26

1 Implementation Details

Our Web Application, **CMMS (Centralized Mess Management System)**, has three major components which are described in detail below.

1.1 Backend

1. **Programming Language:** The backend of CMMS is built using Python. Python's simplicity, readability, and rich library ecosystem make it the ideal choice for building a robust web application. Also while it may not excel in raw execution speed Python's gentle learning curve helped us focus on implementing the project features rather than spending too much time learning the language.

2. **Framework:** We used the Django framework along with Django REST Framework (DRF). Django was chosen for the following reasons:

- **Simple Syntax**: Django is written in Python, making code easy to write and maintain.
- **Built-in Admin Interface**: Django provides a powerful admin panel for CRUD operations without extra code which makes interacting with database and manual testing easier..
- **Object-Relational Mapping (ORM)**: Django's ORM abstracts SQL queries into Python, simplifying database interactions and preventing SQL injection.
- **Robust Security**: Django offers built-in protection against common vulnerabilities such as CSRF(Cross-Site Request Forgery), XSS(Cross Site Scripting), and SQL injection.
- **Rapid Development**: The "batteries-included" philosophy allowed faster feature delivery across authentication, booking, and rebate modules.
- **Model View Controller(MVC) Framework:** While Django does not directly follow MVC, its Model Template View Framework is fairly similar to MVC Framework. Which makes it easier to relate to and work with..
- **Community Support**: A large and active community provided extensive third-party packages and support.

3. **Libraries and Packages**:

- **Django REST Framework (DRF)**: Used to build RESTful APIs consumed by the React frontend. Provides serializers, viewsets, authentication classes, and permission controls.
- **djangorestframework-simplejwt**: Handles JWT (JSON Web Token) generation and verification for secure, stateless authentication of all API endpoints. In our project it is used in HTTP-Only Cookies used for authentication
- **django-cors-headers**: Manages Cross-Origin Resource Sharing (CORS) to allow the React frontend (running on a different port/domain) to communicate with the Django backend.
- **django-anymail / Brevo API**: Integrated with Brevo (formerly Sendinblue) for transactional email delivery, used for OTP-based password reset. We chose Brevo's API over raw SMTP because it eliminates the need to manage SMTP credentials and open mail ports directly, reducing attack surface and simplifying production configuration. The API also provides delivery tracking and handles retries automatically, things you'd otherwise have to build yourself on top of SMTP.

- **WhiteNoise:** Used for serving static files directly from the Django application, eliminating the need for a separate web server like Nginx just to serve CSS, JS, and images in production.
- **ReportLab:** Used for generating PDF bills for students allows us to programmatically create structured, print-ready documents with custom layouts, tables, and formatting directly from Python.
- **python-dotenv:** Used to load environment variables from a .env file, keeping sensitive configuration like API keys, database credentials, and secret keys out of the codebase and secure.

4. **Authentication**:

- User authentication is implemented using JWT tokens.
- Access tokens are short-lived; refresh tokens allow session extension without re-login.
- Tokens are stored as HTTP-only cookies on the client side to mitigate XSS attacks.
- Role-based access control (RBAC) is enforced at the API view level using custom DRF permission classes, ensuring Students, and Admins can only access their permitted endpoints.

1.2 Frontend

1. **Programming Languages and Framework**: The CMMS frontend is built using React.js (bootstrapped with Vite), JavaScript (ES6+), and CSS via TailwindCSS and CSS Animations via Framer-Motion. This stack provides a fast, component-based, and responsive user interface.

2. **Why Vite + React.js:**

- **Vite** provides an extremely fast Hot Module Replacement (HMR) development server meaning if a module is modified it only loads the modified part of code and keeps the rest the same and optimized production builds, significantly improving the developer experience.
- **React.js** enables building a Single Page Application (SPA) with reusable UI components, making the codebase modular and maintainable.
- **React Router** is used for client-side routing, enabling seamless navigation between the Student Dashboard, Booking pages, Rebate pages, and Admin Portal without full page reloads leading to better user experience.

3. **Styling and Animations**:

- **TailwindCSS**: A utility-first CSS framework used for all styling. It allows rapid UI development with consistent design tokens (colors, spacing, typography) and produces a fully responsive layout for both mobile and desktop users.
- **Framer Motion**: Used to implement smooth UI animations and page transitions, improving the overall user experience for example, animated QR code display, card slide-ins, and modal transitions.

4. **Key Frontend Libraries**:

- **axios**: It is an open source framework used for making all asynchronous HTTP requests to the Django REST API. It handles JWT token injection in request headers and provides interceptors for token refresh on expiry. Furthermore we have built our api function on top of it to include and use all our authentication logic.
- **qrcode.react:** A React component library used to dynamically render unique QR codes for confirmed extra item bookings and guest meal tokens, directly in the browser without any server-side rendering.
- **html5-qrcode**: Used in the Mess Staff portal to access the device camera and decode student-presented QR codes for real-time order validation at the mess counter.
- **lucide-react**: Used for scalable vector icons throughout the project provides a clean, consistent icon set as ready-to-use React components without needing to manage raw SVG files manually.
- **React Router DOM**: Manages all client-side routing with protected route components that enforce authentication and role-based redirection.

1.3 Database

For the database, we used **PostgreSQL** in production and **SQLite** for local development. PostgreSQL was chosen for its robustness, ACID compliance, and ability to handle concurrent transactions, a critical requirement for CMMS's inventory management. However since our project uses Django ORM as an abstraction layer, switching to a different database backend requires minimal code changes

Key reasons for choosing PostgreSQL:

- **ACID Compliance**: Guarantees that concurrent booking requests for the same limited extra item are handled correctly preventing double-booking or negative stock.
- **Concurrency Control**: PostgreSQL's row-level locking ensures that only one transaction can decrement the stockCount at a time, preventing race conditions.
- **Scalability**: Handles multiple concurrent student bookings during peak meal hours without performance degradation.
- **Seamless Django ORM Integration**: Django's ORM works natively with PostgreSQL, providing clean model definitions for all entities (User, Booking, ExtraItem, Rebate, Feedback, GuestToken).
- **Data Integrity**: Foreign key constraints and unique constraints are enforced at the database level, ensuring referential integrity for all booking and financial records.
- **SQLite for Development**: Used locally for its zero-configuration setup, making it easy for team members to onboard without a database server installation.

1.4 Building Systems

As CMMS is a full-stack web application with a Django backend and a Vite + React frontend, we utilize the standard toolchains for both.

Backend Build System:

- **Django Management Commands**: The manage.py script is used for running the development server, applying database migrations (makemigrations / migrate), and creating admin superusers.
- **Poetry:** All backend dependencies are managed using Poetry, which handles dependency resolution, version pinning, and virtual environment creation in one tool ensuring consistent builds across developer machines and deployment environments.
-

Frontend Build System:

- **Vite**: Serves as the development server (npm run dev) and production bundler (npm run build). Vite's ES module-based build produces highly optimized static assets.
- **npm**: Used for managing all frontend dependencies defined in package.json.
- **Environment Variables**: Both the backend (.env) and the frontend (.env) use environment variable files to manage secrets and configuration (e.g., database URLs, JWT secrets, API base URLs, Brevo API key).

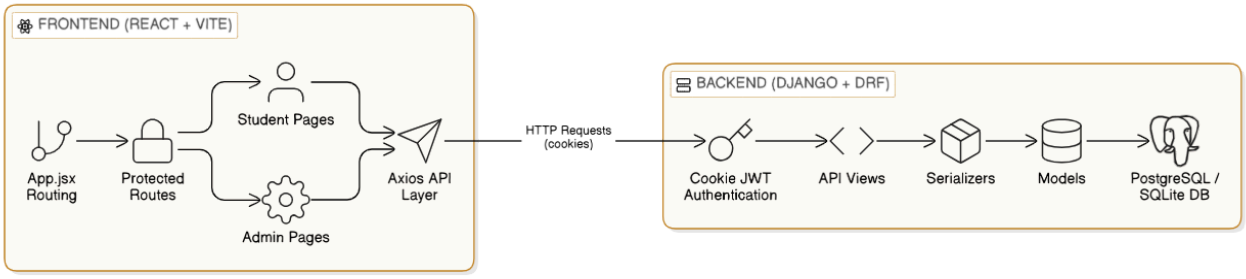
2 Codebase

2.1 Github Repository:

Link : [Chronos193/CMMS: Repo for CS253 project CMMS]

2.2 Codebase Structure:

The CMMS project follows a modular full-stack architecture consisting of two independent repositories: a Django-based backend and a React-based frontend. The interaction between components is illustrated below.



This architecture separates UI rendering, business logic, and data persistence, allowing independent development and deployment of frontend and backend modules.

1. Backend Structure (New-CMMS-Backend)

The backend is implemented using Django and Django REST Framework. It is organised into a project package and a primary application containing business logic.

Core entry and configuration

- manage.py – Django management entry point for running the server and migrations
- requirements.txt – Python dependencies
- CMMS_Backend/settings.py – JWT configuration, CORS, database switching, and rate limiting
- CMMS_Backend/urls.py – Root URL routing

Main application (Backend_App)

- models.py – Database schema including user roles, bookings, menu, rebates, billing, feedback, and notifications
- views.py – API endpoints covering authentication, student features, and admin operations
- serializers.py – Data validation and transformation for all models

- authentication.py – Cookie-based JWT authentication class
- urls.py – Registration of all API routes under /api/
- tests.py – Placeholder for backend tests (currently empty)

The backend handles authentication, business rules, QR-based booking validation, billing generation, rebate processing, and notification delivery.

2. Frontend Structure (New-CMMS-Frontend)

The frontend is built using React with Vite and follows a component-based structure.

Configuration and entry

- package.json – Dependency manifest
- vite.config.js – Build and development configuration
- src/App.jsx – Route definitions and application root

API and routing

- Api.jsx – Centralised Axios instance with cookie authentication and token refresh logic
- ProtectedRoute.jsx – Authenticated route wrapper
- AdminProtectedRoute.jsx – Admin-only route wrapper

Pages

- Student pages: menu viewing, extras booking, cart, rebate, billing, feedback, and bookings
- Admin pages: dashboard, billing management, extras CRUD, rebate approval, notifications, menu management

Reusable components

- NavBar.jsx – Navigation bar with notification and cart indicators
- CartContext.jsx – Global cart state management

The frontend communicates with backend APIs using Axios and renders role-based dashboards for students and administrators.

Together, these two repositories implement a clean separation between the presentation layer, API layer, and persistence layer, enabling scalability and easier maintenance of the CMMS application.

3 Completeness

3.1 Authentication & Authorization

- User registration is implemented with mandatory @iitk.ac.in email validation. Students provide name, roll number, hall of residence, room number, and contact number during signup.
- Login authenticates via email and password and returns JWT tokens (access + refresh) as HttpOnly cookies, mitigating XSS-based token theft. An optional role parameter ensures users log in through the correct portal (student vs. admin).
- A custom CookieJWTAuthentication class extends SimpleJWT to read the access token from the access_token cookie instead of the Authorization header.
- Token refresh is handled via a dedicated endpoint that reads the refresh_token cookie, issues a new access token, and optionally rotates the refresh token (ROTATE_REFRESH_TOKENS is enabled).
- Password reset is implemented using Django's built-in token generator. A reset link is sent via the Brevo transactional email API. The link leads to a server-rendered HTML form where the user sets a new password and is redirected to the frontend login page on success.
- Role-based access control is enforced at the view level: admin-only endpoints check request.user.role == 'admin' and return 403 Forbidden otherwise. Frontend uses ProtectedRoute (checks login) and AdminProtectedRoute (checks login + admin role) components to guard routes.
- Rate limiting is configured: 500 requests/minute for anonymous users, 5000 requests/hour for authenticated users, and 5 requests/minute for the signup endpoint to prevent abuse.

3.2 Mess Menu Display

- The weekly mess menu is fully implemented. Each menu entry consists of a hall, day of the week, meal time (Breakfast, Lunch, Snacks, Dinner), dish name, and optional category.
- Students view the menu for their assigned hall by default. A hall filter allows viewing menus for any hall.
- Admins can add, update, and delete menu items through a dedicated Admin Menu Management page, which calls the backend API endpoints for CRUD operations.

3.3 Notification System

- An in-app notification system is implemented. Notifications are stored in the database with a seen/unseen status and displayed in a dropdown panel in the NavBar.
- Unseen notifications are indicated by a red dot badge on the bell icon. When the dropdown is opened, all unseen notifications are automatically marked as seen via an API call.
- Notifications are triggered automatically by the backend for the following events: rebate application approved/rejected, feedback status updated, bill payment status changed, and QR code scanned (order collected).

- Admins can send custom notifications to individual students (by user ID, email, or roll number), groups of students, or broadcast to all students through the Admin Notification page.
- Each notification displays a title, content, timestamp, and read/unread indicator. Notifications are sorted in reverse chronological order.

3.4 Extra Item Booking & Cart System

- The Extra Meals page displays all available extra items for the student's hall, fetched from the Item and Booking models. Each item shows its name, price, and available stock count.
- Students add items to a persistent server-side cart (Cart model) with quantity selection. The system validates that the requested quantity does not exceed available stock before adding.
- The Cart page shows all cart items with quantities and costs. A cart-check API endpoint re-validates all cart items against current Booking availability, automatically adjusting quantities or removing out-of-stock items.
- Checkout is implemented as an atomic database transaction with row-level locking (select_for_update) to prevent concurrent overselling. The available_count on the Booking is decremented, and MyBooking records are created.
- A single QR code (UUID-based, stored in QRDatabase) is generated per checkout session and shared across all items in that order. The QR code is displayed to the student using the qrcode.react library.
- If an item sells out between browsing and checkout, the system returns an error and the booking is rejected then no charge is applied.
- Students can view their booking history on the My Bookings page, grouped by QR code. Each group shows item details, quantities, total cost, and scan status (Confirmed-Not-Scanned or Confirmed-Scanned).

3.5 QR-Based Order Validation

- A backend API endpoint (AdminQRScanView) is implemented for QR code validation. When a QR code string is submitted, the system looks up the corresponding QRDatabase entry and all associated MyBooking records.
- On successful validation: all unscanned bookings linked to that QR code are updated to 'confirmed-scanned', and the API returns the student's name, email, roll number, item details, quantities, and total cost for display to the mess staff.
- If the QR code has already been scanned, the system returns an 'confirmed_scanned' status with the original order details, preventing duplicate claims.
- If the QR code is invalid or has no associated bookings, appropriate error messages ('Invalid QR code' or 'No bookings found') are returned.
- A notification is automatically sent to the student when their order is marked as collected, showing the number of items and total cost.

- The admin frontend provides a QR scan interface where staff can input or scan QR codes to process orders.

3.6 Rebate Management

- The Mess Rebate Application page allows students to apply for mess leave rebates by selecting a leave start date, end date, and providing a location/reason.
- Eligibility rules are communicated to students on the application page: minimum 3 consecutive days of leave, applications must be submitted at least 2 days before the leave start date, and a maximum of 30 days leave per semester.
- Applications are submitted with status 'Pending'. The Admin Rebate Management page displays all applications and allows admins to approve or reject them with optional notes.
- The rebate refund is calculated automatically: the system determines the number of approved rebate days that overlap with each billing month and multiplies by the per-day refund rate (stored in the DailyRebateRefund model, configurable per month). This amount is deducted from the student's monthly bill.
- In-app notifications are sent to students when their rebate application status is updated to approved or rejected, including the leave period and any admin notes.
- Students can view their complete Application Status History showing all past applications with leave period, duration, submission date, status (Approved/Pending/Rejected), and calculated rebate amount.

3.7 Student Expense Tracking & Mess Bill

- The Mess Bill Summary page provides a consolidated view of all charges for the current or selected month. The bill is organized into clear sections: Fixed Charges (monthly mess fees by category), Extra Item Expenses (itemized with quantities and per-unit costs), Rebate Deductions (days × daily rate), and the Final Payable Amount.
- Monthly expenditure is updated automatically after each successful extra item booking.
- A downloadable PDF mess bill is generated on demand using ReportLab. The PDF includes student details (name, roll number, hall), an itemized charges table, fixed charges breakdown, rebate deductions, and a grand total. Each PDF is stamped with a unique verification UUID (stored in BillVerification) for authenticity verification.
- Payment status tracking is implemented per student per month with statuses: Unpaid, Paid, Overdue, and Waived. Admins can update payment status and the paid_on timestamp is recorded automatically.
- Billing data for all students is accessible to admins through the Admin Billing dashboard, which shows each student's fixed charges, extras, rebate deductions, grand total, and current payment status.

3.8 Feedback and Complaint Management

- The Feedback and Complaint page allows students to submit grievances or suggestions through a structured form. Students select a category from a predefined list (Facilities, Academic, Cafeteria, Food, Hygiene, Library, Administration, Transportation, Technology, or Other) and provide a description.
- Upon submission, the feedback is stored in the database with status 'Pending' and a timestamp is automatically recorded.
- Students can view their previously submitted feedback on the same page with status indicators: Pending, In Progress, and Resolved.
- Administrators can review all student feedback through the Admin Feedback dashboard, which displays feedback cards with student details, category, date, and content. Admins can update the status of each feedback item (Pending → In Progress → Resolved).
- In-app notifications are sent to students when the status of their feedback is updated by the admin, informing them of the new status.
- The Admin Feedback page includes summary statistics showing counts of pending, in-progress, and resolved feedback items.

3.9 Manager / Admin Dashboard

- The Admin Dashboard is implemented as a React-based portal, accessible only to users with the 'admin' role. It serves as a central navigation hub with six management modules: Billing Management, Extras Management, Feedback, Rebate Management, Notification System, and Menu Management.
- **Billing Management:** Admins can view all students' monthly bills with a searchable table showing name, roll number, hall, fixed charges, extras, rebate deductions, grand total, and payment status. Admins can mark bills as Paid, Unpaid, Overdue, or Waived, and send bill payment reminder notifications to individual students.
- **Extras Management:** A full CRUD interface for managing extra meal items. Admins can add new items (with name, price, stock, and hall assignment), edit existing items, delete items, and view live stock counts and sold quantities. The dashboard displays items grouped by hall with an order feed showing recent bookings.
- **Rebate Management:** Lists all student rebate applications with details. Admins can approve or reject applications with notes, triggering automatic notifications to students.
- **Feedback Management:** Lists all student complaints with filtering and status management. Admins update status and the system notifies students of changes.
- **Menu Management:** Admins can add, edit, and delete menu items for any hall, day, and meal time combination through a dedicated interface.
- **Notification System:** A dedicated page for sending custom notifications to individual students (by roll number or email), or broadcasting to all students.

3.10 Deployment & Infrastructure

- The backend is configured for production deployment with WhiteNoise for static file serving, PostgreSQL via dj_database_url for the database, and HTTPS enforcement with SECURE_SSL_REDIRECT.
- Cross-origin requests between the frontend and backend are handled via django-cors-headers with CORS_ALLOW_CREDENTIALS enabled. Cookie SameSite is set to 'None' in production for cross-domain cookie support.
- Environment variables (.env) are used for all secrets and configuration: SECRET_KEY, DATABASE_URL, FRONTEND_URL, BREVO_API_KEY, and DEBUG flag.
- The frontend is built with Vite and served as static assets, with the API base URL configured via VITE_API_URL environment variable.

3.11 Future Implementations

- **Guest Meal Booking:** A planned feature to allow students to book meals for guests, with unique single-use QR tokens tied to a specific date and meal type. This was not implemented in the current version.
- **Analytics Dashboard:** A planned admin feature using Chart.js to display charts for rebate trends, booking volumes, popular extra items, and food demand forecasting. This was not implemented in the current version.
- **Nutritional Information Display:** Displaying calorie, protein, and carbohydrate information alongside menu items. Planned for a future version using manually entered or API-sourced nutritional data.
- **Online Payment Integration:** Integration with a payment gateway (e.g., Razorpay) to enable direct in-app mess bill payments. Currently, the system tracks expenditure and generates bills but payments are processed externally.
- **Push Notifications:** Browser push notifications for real-time booking and rebate status updates are planned to supplement the existing in-app notification system.
- **Backend Rebate Validation:** Server-side enforcement of rebate eligibility rules (minimum 3 days, 2 days advance, 30 days per semester cap). Currently these rules are displayed on the frontend but not enforced by the backend.
- **Cache Implementation:** Frequently accessed API endpoints that return largely static data such as mess menus and fee structures will be cached using Redis, reducing repeated database queries and improving response times.

Appendix A - Group Log

Date and Time	Venue	Activity
11/03/26 9:00 PM	2nd Floor Rajeev Motwani Building	Discussed the structure of the web application. Decided the pages to be made and worked on the front-end.
13/03/26 8:00 PM	4th Floor Rajeev Motwani Building	Discussed about the back-end and the structure of the database.
15/03/26 9:30 PM	4th Floor Rajeev Motwani Building	Divided the work and worked on making the back-end and front-end.
20/03/26 5:00 PM	1st Floor Rajeev Motwani Building	Worked on integrating the front-end with the backend.
24/03/26 9:30 PM	4th Floor Rajeev Motwani Building	Searched for and fixed bugs in the implemented software.
26/03/26 9:00 PM	4th Floor Rajeev Motwani Building	Created Readme files and the Implementation Document for CMMS.